Lab 11: Capstone — End-to-End Investigation

Lab Overview

This capstone has no new commands. Everything you need you have already used; the work here is deciding *which* technique answers each question, and assembling the results into something you could hand to someone else.

You will run a combined security and business review of the Buttercup Games environment, correlate two unrelated data sources, build a dashboard from your findings, and set an alert so the issue is caught next time.

One of the questions has a negative answer. Finding that out *is* the result — ruling out a connection is as much a finding as confirming one.

Learning Objectives

By completing this lab, you will:

- Scope an investigation across three source types
- Extract fields at search time from unstructured log text
- Determine whether two data sources describe the same actor
- Quantify business impact using an enrichment lookup
- Assemble findings into a single dashboard
- Create an alert that would catch a recurrence

Prerequisites

- A running Splunk instance with the course data loaded (completed Lab 1)
- Search, fields and commands (completed Labs 2 through 5)
- Transforming commands (completed Lab 6)
- Reports and dashboards (completed Lab 7)
- Lookups configured (completed Lab 9)
- Alerting (completed Lab 10)

> **Time range**: Use **Last 30 days** throughout unless a step says otherwise.

Scenario

You are the Splunk analyst at Buttercup Games. Overnight, two reports arrived:

1. The web team has seen a rise in **403 Forbidden** responses and suspects someone is probing the storefront.
2. The infrastructure team reports **repeated failed SSH logins** on the web servers.

Leadership wants to know whether these are the same attacker, what the business impact has been, and how the company would detect this faster next time.

Phase 1: Triage the Web Tier

Task 1.1: Size the Problem

Establish how many 403 responses occurred and how they are distributed:

```
index=main sourcetype=access_combined_wcookie status=403
| stats count
```

✓ **Checkpoint**: Record the total.

Task 1.2: Identify the Source

```
index=main sourcetype=access_combined_wcookie status=403
| top limit=5 clientip
```

✓ **Checkpoint**: Record the top client IP and its count.

Task 1.3: Characterise the Behaviour

Determine what that client was reaching for:

```
index=main sourcetype=access_combined_wcookie status=403
| stats count by clientip, file
| sort –count
```

✓ **Checkpoint**: Is the activity concentrated on one resource, or spread across the site?

Phase 2: Triage the Host Tier

Task 2.1: Size the Authentication Failures

```
index=main sourcetype=linux_secure "Failed password"
| stats count
```

Task 2.2: Extract the Attacker and Target

The Linux secure log is unstructured text, so extract the fields you need:

```
index=main sourcetype=linux_secure "Failed password"
| rex "for (?:invalid user )?(?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count by src_ip
| sort -count
```

✓ **Checkpoint**: Record the top source IP and how many distinct source IPs appear in total.

Task 2.3: Identify the Targeted Accounts

```
index=main sourcetype=linux_secure "Failed password"
| rex "for (?:invalid user )?(?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| top limit=5 user
```

✓ **Checkpoint**: Which accounts are being guessed most often?

Task 2.4: Check Whether Anything Succeeded

Start with the totals:

```
index=main sourcetype=linux_secure
| stats count(eval(searchmatch("Failed password"))) as failed,
        count(eval(searchmatch("Accepted password"))) as accepted
```

Totals alone cannot tell you whether an *attacker* succeeded — legitimate administrators log in successfully all day. Break it down by source address:

```
index=main sourcetype=linux_secure
| rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count(eval(searchmatch("Failed password"))) as failed,
        count(eval(searchmatch("Accepted password"))) as accepted by src_ip
| sort -failed
```

✓ **Checkpoint**: Most addresses fall into one of two groups — heavy failures with no successes, or successes with no failures. One address does not. Identify it, and work out what it means.

Task 2.5: Establish What the Intruder Reached

For the address you identified, determine which accounts were successfully used:

```
index=main sourcetype=linux_secure "Accepted password"
| rex "for (?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count by src_ip, user
| sort -count
```

✓ **Checkpoint**: Record the accounts accessed from that address. This is the most serious finding in the investigation.

Phase 3: Correlate the Two Sources

Task 3.1: State the Hypothesis

Leadership assumes one attacker is behind both. Test it rather than assuming it.

Task 3.2: Compare the Address Populations

```
index=main sourcetype=linux_secure "Failed password"
| rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count by src_ip
| rename src_ip as ip
| append
    [search index=main sourcetype=access_combined_wcookie
     | stats count by clientip
     | rename clientip as ip]
| stats dc(count) as appearances by ip
| where appearances > 1
```

Task 3.3: Count Each Population

```
index=main sourcetype=linux_secure "Failed password"
| rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats dc(src_ip) as ssh_sources
```

```
index=main sourcetype=access_combined_wcookie
| stats dc(clientip) as web_clients
```

✓ **Checkpoint**: How many addresses are in each population, and how many appear in both? Write down the conclusion you would report to leadership.

Phase 4: Quantify Business Impact

Task 4.1: Revenue in the Period

Using the products lookup from Lab 9:

```
index=main sourcetype=access_combined_wcookie file=success.do status=200
| stats sum(Price) as Revenue, count as Units
```

Task 4.2: Revenue by Product

```
index=main sourcetype=access_combined_wcookie file=success.do status=200
| stats sum(Price) as Revenue by ProductName
| sort -Revenue
```

Task 4.3: Conversion Health

```
index=main sourcetype=access_combined_wcookie
| stats count(eval(action="addtocart")) as CartAdds,
        count(eval(action="purchase")) as Purchases
| eval ConversionRate=round(Purchases*100/CartAdds,1)
```

✓ **Checkpoint**: Record total revenue, the top product, and the conversion rate. Does the 403 activity appear to have damaged sales?

Phase 5: Build the Investigation Dashboard

Task 5.1: Create the Dashboard

1. Run the search from **Task 1.2**
2. **Save As** → **Dashboard Panel** → **New Dashboard**
3. **Title**: `Security Investigation`
4. **Panel Title**: `Top 403 Sources`

5. Save and view

Task 5.2: Add the Remaining Panels

Add one panel per finding, using **Save As** → **Dashboard Panel** → **Existing**:

Panel Title	Search from
SSH Failures by Source	Task 2.2
Targeted Accounts	Task 2.3
Revenue by Product	Task 4.2
Conversion Rate	Task 4.3

Task 5.3: Choose Sensible Visualizations

Set each panel to a format that suits its data — a column chart for the ranked counts, a single value for the conversion rate.

✓ **Checkpoint**: One dashboard carries both the security and the business findings.

Phase 6: Detect It Next Time

Task 6.1: Create the Alert

Build a scheduled alert for renewed probing:

```
index=main sourcetype=access_combined_wcookie status=403
| stats count by clientip
| where count > 50
```

1. **Save As** → **Alert**
2. **Title**: `Elevated 403 Activity`
3. **Alert type**: Scheduled, run daily
4. **Trigger condition**: Number of Results is greater than 0
5. **Trigger action**: Add to Triggered Alerts, severity High

Task 6.2: Justify the Threshold

Explain why 50 is a defensible threshold for this dataset, referring to the counts you recorded in Phase 1.

✓ **Checkpoint**: The alert is saved and appears under **Activity** → **Triggered Alerts** configuration.

Phase 7: Report

Write a short summary answering:

1. What happened on the web tier, and who was responsible?
2. What happened on the host tier, who was responsible, and did any of it succeed?
3. Are they the same actor? State your evidence.
4. What was the financial picture during the period?
5. What detection did you put in place?

Challenge Exercises

Challenge 1: Attack Timing

Determine whether the SSH attempts cluster at particular times, or run steadily:

```
index=main sourcetype=linux_secure "Failed password"
| timechart span=1d count
```

Does the shape support an automated tool or a person?

Challenge 2: Session Behaviour

The noisiest web client holds far more sessions than any other. Establish whether its sessions look like browsing or like scripted access, using `dc(JSESSIONID)` and the files it requested.

Challenge 3: Rule Out the Error Codes

Determine whether the 403 activity coincides with a rise in 500 errors, which would suggest the probing was destabilising the application.

Challenge 4: Test Whether the Top Offender Is Really an Offender

Phase 1 ranked clients by raw 403 count. That ranking has a weakness: a client that makes more requests than anyone else will tend to top any error ranking.

Normalise the count against each client's total requests:

```
index=main sourcetype=access_combined_wcookie
| stats count(eval(status=403)) as forbidden, count as total by clientip
| eval ratio=round(forbidden*100/total,2)
| sort -forbidden
```

Compare the `ratio` column across clients. Does the client you named in Task 1.2 still look like an attacker? What does your answer imply about the Task 6.1 alert?

Lab Summary

What You Accomplished

- Triaged two independent incidents across three source types
- Extracted fields at search time from unstructured log text
- Tested a correlation hypothesis and reported a negative result
- Quantified revenue and conversion using an enrichment lookup
- Assembled a dashboard spanning security and business findings
- Created a scheduled alert with a justified threshold

Key Concepts Covered

- **Investigation scoping**: narrowing by index, source type and time first
- **Search-time extraction**: `rex` for text that has no fields
- **Correlation**: comparing populations rather than assuming a link
- **Negative findings**: ruling something out is a reportable result
- **Communication**: a dashboard is the deliverable, not the search bar

Where to Go Next

The intermediate course builds directly on this: multiple indexes, knowledge objects, field aliases, calculated fields, macros and data models. The habits practised here — scope first, extract what you need, verify before concluding — carry straight over.

Answer Key

🔑 **Instructor answer key**

Phase 1 — Web Tier

- Task 1.1: **1,936** total 403 responses
- Task 1.2: top client **92.46.53.223** with **350**; then 212.58.253.71 (279) and 91.214.92.22 (238). Treat this ranking as provisional — Challenge 4 revisits it
- Task 1.3: the 403s are spread across resources — product.screen (941), category.screen (639), cart.do (267), success.do (59), oldlink (30) — rather than concentrated on one, so this is not a single broken link

Phase 2 — Host Tier

- Task 2.1: **28,761** failed password events
- Task 2.2: top source **103.99.0.122** with **3,664**; **8** distinct source IPs in total
- Task 2.3: most-targeted accounts are **root** (2,564), **admin** (2,488) and **user** (1,477) — generic high-value names, consistent with a dictionary attack
- Task 2.4: **28,761** failed against **15,900** accepted overall. The per-address breakdown is where the finding is:

Group	Addresses	Pattern
External attackers	103.99.0.122, 94.102.49.190, 198.51.100.42, 167.71.5.83, 45.33.32.156, 185.234.218.110, 202.179.8.245	~3,400–3,700 failures each, **0** successes
Internal administrators	10.0.0.50, 172.16.0.10, 192.168.1.100, 192.168.1.50	**0** failures, ~3,100–3,250 successes each
208.65.153.253	—	**3,628 failures and 3,216 successes**

  `208.65.153.253` is the outlier and the most serious finding: it is the only address that both guessed credentials heavily *and* succeeded. The seven other attackers never got in; the four internal hosts never failed. This address did both, which is the signature of a brute-force attempt that eventually worked

- Task 2.5: successful logins from `208.65.153.253` used **user1** (497), **nsharpe** (475) and **djohnson** (470), among others — real accounts, not the generic `root` / `admin` names being guessed. Treat these accounts as compromised and escalate

Phase 3 — Correlation

- Task 3.2 returns **no rows**
- Task 3.3: **8** SSH source addresses, **12** web client addresses, **zero** in common

- **Conclusion**: the two incidents are unrelated. They share no addresses, and the behaviour differs in kind — credential guessing against a host service versus HTTP probing of the storefront. Report them as two separate issues

Phase 4 — Business Impact

- Task 4.1: **$62,752.46** revenue across **3,054** units
- Task 4.2: top product **Orvil the Wolverine** at **$7,998.00**
- Task 4.3: CartAdds **11,405**, Purchases **3,214**, conversion **28.2%**
- Sales show no dip attributable to the 403 activity — the probing generated errors but did not interrupt purchasing

Phase 6 — Threshold

A threshold of 50 sits below the 238–350 range of the busiest clients, so it will fire. Accept that answer here, but note that Challenge 4 shows why it is the wrong rule: it selects the busiest client rather than a misbehaving one. Students who spot that before reaching Challenge 4 have understood the lesson early.

Challenge 1

`timechart span=1d count` shows a steady daily volume rather than bursts, consistent with an automated tool running continuously.

Challenge 2

The top client holds **3,309** distinct sessions — far above any other client — while requesting the same small set of files. That ratio indicates scripted access, since a human session would revisit fewer sessions across more pages.

Challenge 4

Every client sits between **1.37%** and **1.74%**, including 92.46.53.223 at **1.48%**. There is no outlier.

clientip	forbidden	total	ratio
92.46.53.223	350	23,647	1.48%
212.58.253.71	279	19,981	1.40%
91.214.92.22	238	16,796	1.42%
116.159.208.78	111	6,364	1.74%
10.0.0.50	14	1,411	0.99%

So 92.46.53.223 leads the raw 403 count simply because it is the **busiest** client, not because it behaves differently. The 403s are distributed in proportion to traffic, which points at an application or permissions issue affecting everyone rather than a targeted attack.

This undercuts the Task 6.1 alert: a threshold on raw count will fire on whoever happens to be busiest, which is a false positive by design. A better rule alerts on a client whose ratio departs from the population, for example:

```
index=main sourcetype=access_combined_wcookie
| stats count(eval(status=403)) as forbidden, count as total by clientip
| eval ratio=round(forbidden*100/total,2)
| eventstats avg(ratio) as avg_ratio
| where forbidden > 50 AND ratio > avg_ratio * 2
```

Against this dataset that correctly returns **nothing**, because no client is anomalous. An alert that stays quiet when nothing is wrong is working.